

GUIDE D'INSTALLATION ET D'UTILISATION - SAE 3.02

Routage en Oignon Distribué avec Anonymisation

Élève : Adame Bouhssine

TABLE DES MATIÈRES

Prérequis système

Installation

Configuration

Lancement du projet

Guide d'utilisation

Dépannage

PRÉREQUIS SYSTÈME

Configuration matérielle minimale

RAM : 2 GB minimum (4 GB recommandé)

Processeur : Dual-core 2.0 GHz minimum

Disque : 500 MB d'espace libre

Réseau : Connexion locale ou LAN disponible

Logiciels requis

Python 3.8+

Vérifier la version installée

`python3 --version`

Système d'exploitation supporté

Linux (Debian, Ubuntu, Fedora)

Base de données

MariaDB 10.5+ ou MySQL 8.0+

Accès root ou privilèges de création de base de données

INSTALLATION

Étape 1 : Cloner ou télécharger le projet

Télécharger manuellement et extraire

```
unzip sae_3_02.zip  
cd sae_3_02
```

Étape 2 : Installer les dépendances Python

Créer un environnement virtuel

```
python3 -m venv venv
```

Activer l'environnement virtuel

Sur Linux :

```
source venv/bin/activate
```

Installer depuis requirements.txt

```
pip install -r requirements.txt
```

Ou installer manuellement :

```
pip install PyQt5
```

```
5.15.7pip install mysql-connector-python8.0.33
```

Fichier requirements.txt (à créer s'il n'existe pas) :

```
PyQt5
```

```
5.15.7mysql-connector-python8.0.33
```

Étape 3 : Installer et configurer MariaDB/MySQL

Sur Linux (Debian/Ubuntu)

Installer MariaDB

```
sudo apt-get update  
sudo apt-get install mariadb-server
```

Démarrer le service

```
sudo systemctl start mariadb  
sudo systemctl enable mariadb
```

Accéder à la console MySQL

```
sudo mysql -u root
```

Ou télécharger depuis mariadb.org

Étape 4 : Créer la base de données

Se connecter à MySQL

```
mysql -u root -p
```

Créer la base de données

```
CREATE DATABASE sae_onion;
```

Créer un utilisateur dédié (optionnel mais recommandé)

```
CREATE USER 'sae_user'@'localhost' IDENTIFIED BY  
'sae_password_123';  
GRANT ALL PRIVILEGES ON sae_onion.* TO  
'sae_user'@'localhost';  
FLUSH PRIVILEGES;
```

Quitter

```
EXIT;
```

Étape 5 : Initialiser les tables

À partir du répertoire du projet

```
mysql -u sae_user -p sae_onion < database_init.sql
```

Ou via Python (voir section Configuration)

```
python3 database.py --init
```

Fichier database_init.sql :

```
CREATE TABLE IF NOT EXISTS routeurs (  
id INT PRIMARY KEY AUTO_INCREMENT,  
ip VARCHAR(15) NOT NULL,  
port INT NOT NULL,  
public_key TEXT NOT NULL,  
status VARCHAR(20) DEFAULT 'active',  
created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
UNIQUE KEY unique_router (ip, port)  
);
```

```
CREATE TABLE IF NOT EXISTS logs (  
id INT PRIMARY KEY AUTO_INCREMENT,  
router_id INT,  
action VARCHAR(50) NOT NULL,  
details TEXT,  
timestamp TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
INDEX idx_router_id (router_id),  
INDEX idx_timestamp (timestamp),  
FOREIGN KEY (router_id) REFERENCES routeurs(id) ON  
DELETE CASCADE  
);
```

```
CREATE TABLE IF NOT EXISTS routes (  
id INT PRIMARY KEY AUTO_INCREMENT,  
source VARCHAR(50) NOT NULL,  
destination VARCHAR(50) NOT NULL,  
next_hop VARCHAR(50) NOT NULL,  
interface VARCHAR(20),  
created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

CONFIGURATION

Fichier de configuration principal

Créer un fichier config.py dans le répertoire du projet :

config.py - Configuration du projet SAE 3.02

Configuration du Master

MASTER_HOST = '0.0.0.0'

MASTER_PORT = 8000

Configuration des routeurs

ROUTER_HOST = '0.0.0.0'

ROUTER_PORTS = [9001, 9002, 9003]

Configuration des clients

CLIENT_HOST = '127.0.0.1'

CLIENT_PORTS = [10000, 10001]

Configuration de la base de données

```
DB_CONFIG = {  
'host': 'localhost',  
'user': 'sae_user',  
'password': 'sae_password_123',  
'database': 'sae_onion',  
'charset': 'utf8mb4',  
'use_unicode': True  
}
```

Configuration réseau

NETWORK_TIMEOUT = 10 # secondes

SOCKET_BUFFER_SIZE = 8192 # bytes

MAX_CONNECTIONS = 10

Configuration cryptographie

CRYPTO_KEY_LENGTH = 64 # caractères

ADDRESS_ENCRYPTED_SIZE = 20 # bytes

Configuration logging

LOG_LEVEL = 'INFO' # DEBUG, INFO, WARNING, ERROR

LOG_FILE = 'sae_onion.log'

Configuration des variables d'environnement

Créer un fichier `.env` (optionnel) :

.env - Variables d'environnement

```
DB_HOST=localhost
DB_USER=sae_user
DB_PASSWORD=sae_password_123
DB_NAME=sae_onion
MASTER_PORT=8000
DEBUG=False
```

Charger les variables dans Python :

```
from dotenv import load_dotenv
import os

load_dotenv()

DB_HOST = os.getenv('DB_HOST', 'localhost')
DB_USER = os.getenv('DB_USER', 'root')
DB_PASSWORD = os.getenv('DB_PASSWORD', '')
```

LANCEMENT DU PROJET

Architecture de lancement

Le projet utilise une architecture **Master-Router-Client** :

Master (port 8000)

↓

Router1 (9001) ← enregistrement

Router2 (9002) ← enregistrement

Router3 (9003) ← enregistrement

↓

Client A (10000) → récupère liste routeurs

Client B (10001) → reçoit messages

Lancement manuel (terminal)

Terminal 1 : Démarrer le Master

```
python3 master.py
```

Output attendu :

```
[2025-12-31 14:00:00] Master démarré sur 0.0.0.0:8000  
[2025-12-31 14:00:00] En attente de connexions...
```

Terminal 2 : Démarrer Router 1

```
python3 router.py --id 0 --port 9001
```

Output attendu :

```
[2025-12-31 14:00:05] Router 0 initialisé (port 9001)  
[2025-12-31 14:00:05] Enregistrement au Master...  
[2025-12-31 14:00:05] ✓ Enregistré (ID: 0)  
[2025-12-31 14:00:05] En attente de messages...
```

Terminal 3 : Démarrer Router 2

```
python3 router.py --id 1 --port 9002
```

Terminal 4 : Démarrer Router 3

```
python3 router.py --id 2 --port 9003
```

Terminal 5 : Lancer le Master GUI (optionnel)

```
python3 master_gui.py
```

Terminal 6 : Lancer Client GUI

```
python3 client_gui.py --id ClientA
```

Une fenêtre PyQt5 apparaît pour l'interface client.

Lancement automatisé (script)

Créer un fichier `launch.sh` :

```
#!/bin/bash
```

launch.sh - Script de lancement du projet SAE 3.02

```
set -e # Arrêter en cas d'erreur
```

```
PROJECT_DIR="$(cd"(dirname "PROJECT_DIR/venv"  
LOG_DIR="$PROJECT_DIR/logs"
```

Créer le répertoire logs

```
mkdir -p "$LOG_DIR"
```

Activer l'environnement virtuel

```
if [ ! -d  
"  
$ENV_PATH ]; then echo "Créating virtual environment..." python3 -  
mvenv "$ENV_PATH"  
fi  
  
source "$ENV_PATH/bin/activate"  
  
echo "Environnement activé"  
echo "Démarrage du Master..."
```

Démarrer le Master en arrière-plan

```
python3 "$PROJECT_DIR/master.py" > "LOG_DIR/master.log"  
2>&1 &  
MASTER_PID=$!  
echo "Master PID: $MASTER_PID"  
  
sleep 2
```

Démarrer les 3 routeurs en arrière-plan

```
for i in 0 1 2; do  
PORT=$((9001 + i))  
echo "Démarrage du Router $i (port  
$PORT)... "python3 "$PROJECT_DIR/router.py" --id $i --port  
$PORT > "LOG_DIR/router_$i]=$!  
echo "Router $i PID: $!"  
sleep 1  
done  
  
echo ""  
echo "=
```

```
"echo "Projet SAE 3.02 démarré"echo "Master PID:
```



```
$MASTER_PID"echo "Router PIDs: ${ROUTER_PIDS[@]}"echo  
""echo "Logs disponibles dans: $LOG_DIR"echo "Lancer le Client  
GUI: python3 client_gui.py"echo "="  
echo ""
```

Optionnel : Afficher les logs en temps réel

```
tail -f "$LOG_DIR/master.log"
```

Rendre le script exécutable et lancer :

```
chmod +x launch.sh  
./launch.sh
```

Lancement avec Docker (optionnel)

Créer un Dockerfile :

```
FROM python:3.9-slim
```

```
WORKDIR /app
```

Installer les dépendances système

```
RUN apt-get update && apt-get install -y  
mariadb-client  
&& rm -rf /var/lib/apt/lists/*
```

Copier le projet

```
COPY . .
```

Installer les dépendances Python

```
RUN pip install --no-cache-dir -r requirements.txt
```

Exposer les ports

```
EXPOSE 8000 9001 9002 9003 10000 10001
```

Lancer le Master par défaut

```
CMD ["python3", "master.py"]
```

Construire et lancer :

Construire l'image

```
docker build -t sae_onion:latest .
```

Lancer le Master

```
docker run -p 8000:8000 --name master sae_onion:latest python3 master.py
```

Lancer un routeur (dans un autre terminal)

```
docker run -p 9001:9001 --link master:master sae_onion:latest python3 router.py --id 0 --port 9001 --master-host master
```

GUIDE D'UTILISATION

Utilisation du Master (CLI)

Options de ligne de commande

```
python3 master.py [OPTIONS]
```

Options disponibles :

Option	Description	Défaut
<code>--host HOST</code>	Adresse d'écoute	<code>0.0.0.0</code>
<code>--port PORT</code>	Port d'écoute	<code>8000</code>
<code>--db-host HOST</code>	Hôte de la base de données	<code>localhost</code>
<code>--db-user USER</code>	Utilisateur BDD	<code>root</code>
<code>--db-password PASS</code>	Mot de passe BDD	<code>""</code>
<code>--db-name NAME</code>	Nom de la base de données	<code>sae_onion</code>

<code>--gui</code>	Lancer avec interface graphique	False
<code>--log-level LEVEL</code>	Niveau de log (DEBUG/INFO/WARNING)	INFO
<code>--version</code>	Afficher la version	-
<code>--help</code>	Afficher l'aide	-

Exemples :

Lancement standard

```
python3 master.py
```

Avec interface graphique

```
python3 master.py --gui
```

Configuration personnalisée

```
python3 master.py --port 8001 --db-user sae_user --db-password secret --log-level DEBUG
```

Vérifier la version

```
python3 master.py --version
```

Utilisation des Routeurs

Options de ligne de commande

```
python3 router.py [OPTIONS]
```

Options disponibles :

Option	Description	Défaut
<code>--id ID</code>	ID unique du routeur	0
<code>--port PORT</code>	Port d'écoute	9001
<code>--host HOST</code>	Adresse d'écoute	0.0.0.0

<code>--master-host</code> HOST	Hôte du Master	127.0.0.1
<code>--master-port</code> PORT	Port du Master	8000
<code>--log-level</code> LEVEL	Niveau de log	INFO
<code>--help</code>	Afficher l'aide	-

Exemples :

Routeur 0 sur port 9001

```
python3 router.py --id 0 --port 9001
```

Routeur 1 sur port 9002, Master distant

```
python3 router.py --id 1 --port 9002 --master-host 192.168.1.100 --master-port 8000
```

Routeur 2 avec logs détaillés

```
python3 router.py --id 2 --port 9003 --log-level DEBUG
```

Utilisation du Client (CLI)

Options de ligne de commande

```
python3 client.py [OPTIONS]
```

Options disponibles :

Option	Description	Défaut
<code>--id</code> ID	ID du client	ClientA
<code>--master-host</code> HOST	Hôte du Master	127.0.0.1
<code>--master-port</code> PORT	Port du Master	8000
<code>--mode</code> MODE	Mode de routage (random/manual)	random

<code>--message MSG</code>	Message à envoyer	-
<code>--recipient RCPT</code>	Client destinataire	-
<code>--log-level LEVEL</code>	Niveau de log	INFO
<code>--help</code>	Afficher l'aide	-

Exemples :

Mode interactif

```
python3 client.py --id ClientA
```

Envoyer un message en direct

```
python3 client.py --id ClientA --recipient ClientB --message
"Bonjour!" --mode random
```

Mode manuel (choisir les routeurs)

```
python3 client.py --id ClientA --mode manual
```

Destinataire distant

```
python3 client.py --id ClientA --master-host 192.168.1.100
```

Utilisation du Client GUI

Lancement

```
python3 client_gui.py --id ClientA
```

Interface graphique

Zone supérieure :

Destinataire : ComboBox pour sélectionner le client destinataire

Mode routage : RadioButton pour choisir Aléatoire/Manuel

Zone centrale :

Message : TextEdit pour saisir le message à envoyer

Bouton Envoyer : Lance l'envoi du message

Zone inférieure :

Logs : Affichage en temps réel des événements

[INFO] Récupération liste routeurs...

[INFO] Message construit (65 bytes)

[INFO] Envoyé à Router1:9001

[INFO] Message reçu: Bonjour!

Flux de travail Client GUI

Au démarrage :

Connexion au Master

Récupération de la liste des routeurs

Affichage dans le ComboBox destinataire

Envoi d'un message :

Saisir le texte dans le champ Message

Sélectionner le destinataire

Choisir le mode de routage (Aléatoire par défaut)

Cliquer sur "Envoyer"

Réception :

Le Client GUI accepte les messages entrants

Affichage dans la zone logs

Message apparaît en notification

Options du Client GUI

Menu **Fichier** :

Paramètres : Configurer hôte Master, port, ID client

Quitter : Fermer l'application

Menu **Affichage** :

Rafraîchir routeurs : Recharger la liste des routeurs

Logs détaillés : Activer les logs DEBUG

À propos : Informations sur l'application

Utilisation du Master GUI

Lancement

python3 master_gui.py

Ou via le Master CLI :

python3 master.py --gui

Interface graphique

Tableau supérieur - Routeurs actifs :

ID, IP, Port, Clé Publique, Status, Timestamp d'enregistrement

Rafraîchissement automatique chaque 2 secondes

Zone centrale - Logs d'événements :

Enregistrements de routeurs

Déconnexions

Erreurs de communication

Boutons de contrôle :

Start : Démarrer le serveur Master

Stop : Arrêter le serveur proprement

Refresh : Rafraîchir les données

Clear Logs : Effacer l'historique

Compteurs :

Nombre de routeurs actifs

Nombre total de messages traités

Nombre d'erreurs

DÉPANNAGE

Problèmes courants et solutions

Erreur : Connection refused au démarrage du routeur

Cause : Le Master n'est pas démarré ou n'écoute pas sur le bon port

Solution :

Vérifier que le Master écoute

```
netstat -tulpn | grep 8000
```

Redémarrer le Master

```
python3 master.py --host 0.0.0.0 --port 8000
```

Erreur : Access denied for user 'root'@'localhost'

Cause : Identifiants de base de données incorrects

Solution :

Vérifier les identifiants dans config.py

Se reconnecter à MySQL

```
mysql -u root -p
```

Vérifier l'utilisateur créé

```
SELECT user, host FROM mysql.user;
```

Réinitialiser le mot de passe si nécessaire

```
ALTER USER 'sae_user'@'localhost' IDENTIFIED BY 'new_password';
```

Erreur : Port already in use

Cause : Un processus occupe déjà le port

Solution :

Trouver le processus qui utilise le port

```
lsof -i :9001
```


Terminer le processus (PID à remplacer)

`kill -9 <PID>`

Ou changer de port

`python3 router.py --id 0 --port 9010`

Erreur : ModuleNotFoundError: No module named 'PyQt5'

Cause : Les dépendances Python ne sont pas installées

Solution :

Activer l'environnement virtuel

`source venv/bin/activate`

Réinstaller les dépendances

`pip install -r requirements.txt`

Ou installer PyQt5 directement

`pip install PyQt5==5.15.7`

Client ne reçoit pas de messages

Cause : Le Client n'écoute pas ou ne s'est pas enregistré

Solution :

Vérifier que le Client est lancé

`python3 client.py --id ClientB`

Vérifier les logs

`tail -f sae_onion.log | grep ClientB`

Vérifier la connectivité réseau

`ping 127.0.0.1`

Routeur n'apparaît pas dans la liste du Master

Cause : L'enregistrement au Master a échoué

Solution :

Vérifier les logs du routeur

`tail -f logs/router_o.log`

Vérifier que le Master écoute

`python3 master.py --log-level DEBUG`

Redémarrer le routeur

`python3 router.py --id o --port 9001 --log-level DEBUG`

Commandes de diagnostic utiles

Vérifier les ports ouverts

Linux

`lsof -i -P -n | grep LISTEN`

Tester la connectivité

Tester le Master

`telnet 127.0.0.1 8000`

Tester un routeur

`telnet 127.0.0.1 9001`

Consulter les logs

Afficher les logs en temps réel

`tail -f sae_onion.log`

Afficher les dernières 100 lignes

`tail -100 sae_onion.log`

Chercher une erreur spécifique

`grep ERROR sae_onion.log`

Afficher les logs par composant

```
grep Router sae_onion.log  
grep Master sae_onion.log  
grep Client sae_onion.log
```

Nettoyer les processus

Tuer tous les processus Python du projet

```
pkill -f "python3 master.py"  
pkill -f "python3 router.py"  
pkill -f "python3 client.py"
```

Ou plus agressif (attention!)

```
pkill -9 python3
```

Vérification de la base de données

Se connecter à la base

```
mysql -u sae_user -p sae_onion
```

Vérifier les routeurs enregistrés

```
SELECT * FROM routeurs;
```

Voir les logs des routeurs

```
SELECT * FROM logs ORDER BY timestamp DESC LIMIT 20;
```

Voir les routes actives

```
SELECT * FROM routes;
```

Vider les tables (ATTENTION!)

```
TRUNCATE TABLE routeurs;  
TRUNCATE TABLE logs;  
TRUNCATE TABLE routes;
```

ANNEXES

Annexe A : Structure des fichiers du projet

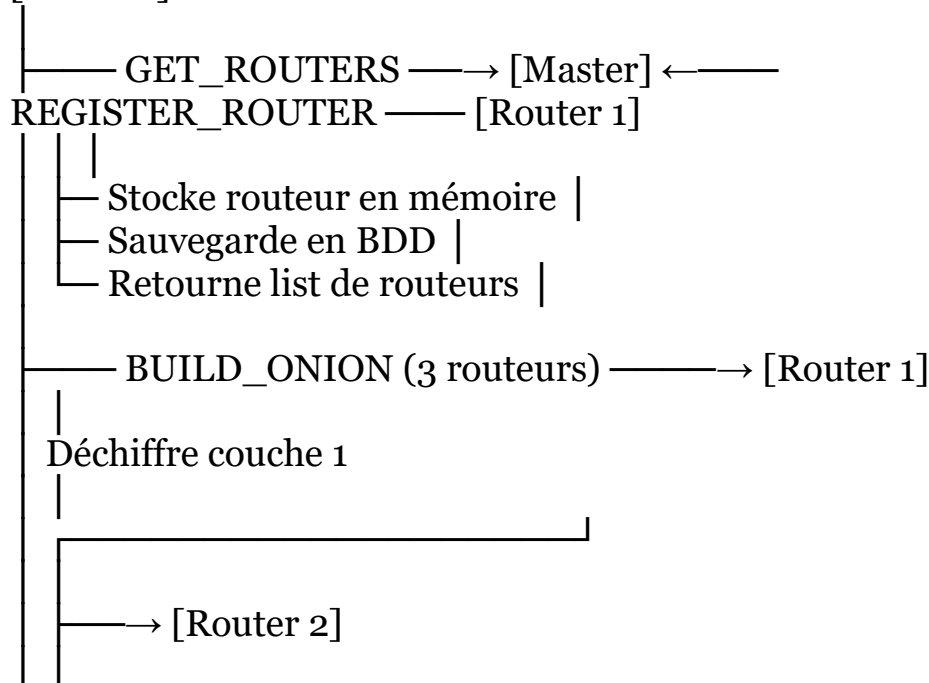
```

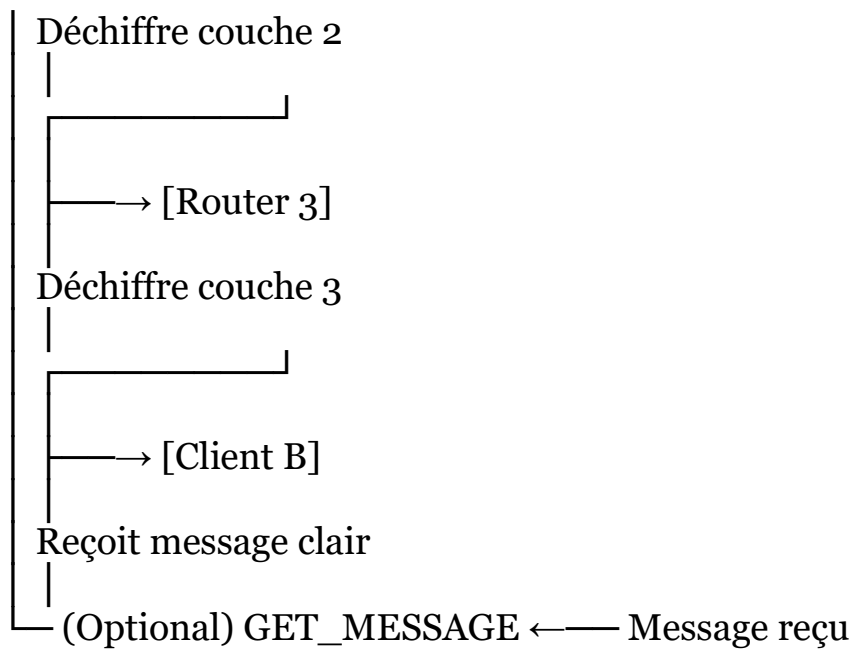
sae_3_02/
├── master.py # Serveur Master
├── router.py # Routeur virtuel
├── client.py # Client CLI
├── client_gui.py # Client GUI (PyQt5)
├── master_gui.py # Master GUI (PyQt5)
├── database.py # Module base de données
├── crypto_simple.py # Module cryptographie
├── config.py # Configuration du projet
├── requirements.txt # Dépendances Python
├── database_init.sql # Initialisation BDD
├── launch.sh # Script de lancement
├── Dockerfile # Configuration Docker
├── .env # Variables d'environnement
├── logs/ # Répertoire des logs
│   ├── master.log
│   ├── router_0.log
│   ├── router_1.log
│   └── router_2.log
├── venv/ # Environnement virtuel Python
└── README.md # Documentation

```

Annexe B : Schéma de communication

[Client A]





Annexe C : Exemples de scripts d'intégration

Script de test automatisé (test_system.sh) :

```
#!/bin/bash

echo "Testing SAE 3.02 Onion Routing System"
echo "=====
```

Vérifier Master

```
echo "Test 1: Master connectivity..."
python3 -c "import socket; s = socket.socket();
s.connect(('127.0.0.1', 8000)); print('✓ Master OK'); s.close()"
2>/dev/null || echo "X Master non disponible"
```

Vérifier Routeurs

```
for i in 0 1 2; do
port=$((9001 + i))echo"Test2.((i+1)): Router $i on port $port..."
python3 -c "import socket; s = socket.socket();
s.connect(('127.0.0.1', $port)); print('✓ Router $i OK'); s.close()"
2>/dev/null || echo "X Router $i non disponible"
done
```

Vérifier BDD

```
echo "Test 3: Database connectivity..."
mysql -u sae_user -p -e "SELECT COUNT(*) FROM
sae_onion.routeurs;" 2>/dev/null && echo "✓ Database OK" ||
echo "X Database non disponible"

echo ""
echo "Tests complétés"
```

Fin du guide d'installation et d'utilisation